

Cryptography Homework 7

Piter Zac. Garcia B.

Vasudev Prasad Bethamcherla

1. Show that decryption is the reverse of encryption for the following Feistel-like cryptosystem (Exercise 4.9.8). Divide the plaintext into three blocks L_i , M_i , and R_i , and let the i th round key be K_i . Compute the next round blocks as follows.

$$L_i = R_{i-1}, M_i = L_{i-1}, R_i = f(K_i, R_{i-1}) \oplus M_{i-1}$$

The decryption algorithm will be as follows, starting with the cipher-text A_n, B_n, C_n :

$$A_{i-1} = B_i, B_{i-1} = f(K_i, A_i) \oplus C_i, C_{i-1} = A_i$$

Show that $A_i, B_i, C_i = L_i, M_i, R_i$ for all i .

- First, divide the plaintext into three equal blocks L_0, M_0, R_0 , where i^{th} is the encryption round obtained by $L_i = R_{i-1}; M_i = L_{i-1}; R_i = f(K_i; R_{i-1}) \oplus M_{i-1}$.

- Next, Let $i^{th} = K_i$, and let f = appropriate size output function. Then, let's consider a decryption for a cipher-text A_n, B_n, C_n using the following algorithm:

$A_{i-1} = B_i; B_{i-1} = f(K_i, A_i) \oplus C_i; C_{i-1} = A_i$, and i goes from n rounds to A_0, B_0, C_0 . This shows that $A_i = L_i, B_i = M_i, C_i = R_i$, for all i , the decryption algorithm returns the plaintext.

- Finally, we observe the relationship between the encryption and decryption processes. Encryption involves a right shift and decryption involves a left shift. So, the i^{th} encryption step is the same as the $n - i^{th}$ decryption step. Therefore, it is shown that the encryption and decryption functions undo each other as following:

Consider the n th round of encryption:

$$[L_{n-1}][M_{n-1}][R_{n-1}] \rightarrow [R_{n-1}][L_{n-1}][f(K_n, R_{n-1}) \oplus M_{n-1}] = [L_{n-1}][M_{n-1}][R_{n-1}] = [A_n][B_n][C_n]$$

$$\text{Decrypting we see that, } [A_n][B_n][C_n] \rightarrow [B_n][f(K_n, A_n) \oplus C_n][A_n] = [L_{n-1}][f(K_n; R_{n-1}) \oplus f(K_n; R_{n-1}) \oplus [M_{n-1}][R_{n-1}]] = [L_{n-1}][M_{n-1}][R_{n-1}]$$

Therefore, each round of decryption gives the prior round after encryption.

As a result it gives $A_i = L_i, B_i = M_i, C_i = R_i$.

1. Compute the output of the AES S-box on input 10110010 by working through the mathematical definition of the S-box. That is, view this input as an element of $GF(2^8)$, compute the inverse of this field element using the extended Euclidean algorithm, view this element as a bit vector, multiply this vector by the matrix on page 157, and add vector (1,1,0,0,0,1,1,0). View the resulting vector as a byte. This is the output of the S-box. Give all your calculations.

The multiplicative inverse for an element of a finite field can be calculated by multiplying a by every number in the field until the product is one. The following is an example calculation of how the input is multiplied to the Galois field and mod, divided, to the Galois field.

1 0 0 0 1 1 0 1	right shift	1 0 0 1 0 1 0 1
0 1 0 1 1 0 0 1	1 0 0 0 1 1 0 1	sum = 1
sum = 1	0 1 0 1 0 1 1 0	
1)	sum = 1	right shift
left shift		1 0 0 0 1 1 0 1
1 0 0 0 1 1 0 1	3)	1 0 0 1 0 1 0 1
1 0 1 1 0 0 1 0	left shift	sum = 2
sum = 1	1 0 0 0 1 1 0 1	left shift = right shifts
	1 1 0 0 1 0 1 0	
	sum = 0	5)
right shift		left shift
1 0 0 0 1 1 0 1	right shift	1 0 0 0 1 1 0 1
1 0 1 0 1 1 0 0	1 0 0 0 1 1 0 1	0 0 1 0 1 0 1 1
sum = 1	0 0 1 0 1 0 1 1	sum = 0
2)	sum = 0	
left shift		right shift
1 0 0 0 1 1 0 1	4)	1 0 0 0 1 1 0 1
0 1 1 0 0 1 0 1	left shift	1 1 0 0 1 0 1 0
sum = 1	1 0 0 0 1 1 0 1	sum = 0

The above operation was done eight times, avoiding 4) since the left and right shift resulted to be the same input. This is how it was approached, if the rightmost bit is set, exclusive OR the product with the value to perform the polynomial addition.

- Shift one bit to the right, discarding the rightmost bit to divide the polynomial by the x^0 term, keeping track of whether the leftmost bit, the carry, is set to one.

- Shift a one bit to the left, discarding the leftmost bit to multiply the polynomial by the x -term, keeping track of the carry, which represented the coefficient of x^7 .

- Finally, the high term of the irreducible polynomial and carry add modulo 2 to 0.

	Shifting Metrix	Carry	XOR
1	0 1 0 1 1 0 0 1	0 0	1
2	1 0 1 1 0 0 1 0	1 1	1
3	1 0 1 0 1 1 0 0	1 1	1
4	0 1 1 0 0 1 0 1	0 0	1
5	0 1 0 1 0 1 1 0	0 0	1
6	1 1 0 0 1 0 1 0	1 0	0
7	0 0 1 0 1 0 1 1	1 0	0
8	1 0 0 1 0 1 0 1 1 0 0 1 0 1 0 1	1 1 1 1	0

1	0	0	0	1	0	0	0	1	1	1	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1	1	0	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	0	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The output of achieved as follows:

First, once the inverse was calculated, as shown above, we took this and multiplied it with the matrix, as explained below, to obtain the desired output.

Second, we took each row of the first matrix, call this matrix A, and multiply it by the first row of the first vector, call this v1, and then xor all resulting product. Keep on doing the same approach from row 1th till row 8th.

Finally, this yields the byte 10110010. The first 4 bits 1011 represent 11 in binary and the last 4 bits 0010 represent 2 in binary. We added 1 to each of these numbers, since the first row and column are numbered 0, and looked in the 12th row and 3rd column of the S-box. The entry is 55, which in binary is 11111000. In conclusion, the matrix chosen mostly has a simple form because its rows are shifts of each other. Also, the vector chosen allows no input to equal the S-box output or its complement. Some of the considerations in the design of the S-box were the map $x \rightarrow x^{-1}$, which was used to achieve nonlinearity. The simplicity of this map could possibly allow certain attacks since it was combined with multiplication by the matrix and adding the vector as described previously.

JAVA CODE :

```
public class GaloisF {
    static ArrayList<Integer> remainder = new
    ArrayList<Integer>();
    static ArrayList<Integer> quotient = new
    ArrayList<Integer>();
    static ArrayList<Integer> auxiliary = new
    ArrayList<Integer>();

    public static void main( String[] args){
        String fx = "100011011";
        String input = "0101100110";

        remainder.add(binaryToDecimal(fx));
        remainder.add(binaryToDecimal(input));

        auxiliary.add(0);
        auxiliary.add(1);
        quotient.add(0);
        quotient.add(0);

        System.out.println(binaryToDecimal(fx));
        System.out.println(binaryToDecimal(input));

        getInverse();
        System.out.println(binaryToDecimal("01101010
        "));
        byte a = (byte) 100011011;
        byte b = (byte) 011001101;

        int f=binaryToDecimal(fx);
        int in = binaryToDecimal(input);

        int x = 0, y= 0 ;

        do{
            x = new Random().nextInt(f);

            } while( (in*x)%f != 1%f);

            System.out.println( "x = " + x);
            System.out.println( "x = " +
            decimalToBinary(x));

            System.out.println(galoisMultiply(a, b));
            System.out.println(decimalToBinary(Math.abs(g
            aloisMultiply(a, b))));
            System.out.println(FFMul(a, FFMul(a, b)));
            System.out.println(decimalToBinary(Math.abs(F
            FMul(a, FFMul(a, b))));
            System.out.println(GMul(a, b));
        }

        public static void getInverse(){
            int i = 1;

            while( remainder.get(i) > 1){
                i++;
                remainder.add(i, remainder.get(i-
                2)%remainder.get(i-1));

                System.out.println( "Remainder " +
                decimalToBinary(Math.abs(remainder.get(i))));
                quotient.add(i, remainder.get(i-
                2)/remainder.get(i-1));
                System.out.println("Quotient " +
                decimalToBinary(Math.abs(quotient.get(i))));aux
                iliary.add(i, -quotient.get(i)*auxiliary.get(i-1)+
                auxiliary.get(i-2));
                System.out.println("auxiliary " +
                decimalToBinary(Math.abs(auxiliary.get(i))));
                System.out.println();
            }

            //System.out.println(auxiliary.get(i));
            System.out.println(decimalToBinary(Math.abs(a
            uxiliary.get(i))));
        }
    }
}
```

```

public static String decimalToBinary(int
decimal) {
return Integer.toBinaryString(decimal);
}

public static int binaryToDecimal(String bin){

char[] binary = bin.toCharArray();
        int power = 0;
        int base = 2;
        int answer = 0;
        for(char ch: binary){

            if( ch == '1'){
answer += Math.pow(base, power);

                power++;
            }else{
                power++;
            }
        }
        return answer;
    }

private static int galoisMultiply(byte a, byte b)
{

    int p = 0;
    for (int n=0; n<8; n++) {

        p = ((b & 0x01) > 0) ? p^a : p;
        boolean ho = ((a & 0x80) > 0);

        a = (byte) ((a<<1) & 0xFE);
        if (ho)
            a = (byte) (a ^ 0x1b);
        b = (byte) ((b>>1) & 0x7F);
    }
    return p;
}

public static byte FFMul(byte a, byte b) {
    byte aa = a, bb = b, r = 0, t;
    while (aa != 0) {
        if ((aa & 1) != 0)
            r = (byte) (r ^ bb);
        t = (byte) (bb & 0x80);
        bb = (byte) (bb << 1);
        if (t != 0)
            bb = (byte) (bb ^ 0x1b);
        aa = (byte) (aa >> 1);
    }
    return r;
}

```

MATLAB CODE:

```

v = [1; 1; 0; 0; 0; 1; 1; 0];
A = [1 0 0 0 1 1 1 1; 1 1 0 0 0
1 1 1; 1 1 1 0 0 0 1 1; 1 1 1 1
0 0 0 1;
      1 1 1 1 1 0 0 0; 0 1 1 1 1 1
0 0; 0 0 1 1 1 1 1 0; 0 0 0 1 1
1 1 1];

B = [0; 1; 1; 1; 0; 1; 1; 1]
B = [1; 1; 1; 1; 1; 0; 0; 0]
% B = [0; 0; 1; 0; 1; 0; 1; 0]

C = [1; 1; 0; 0; 0; 1; 1; 0]
S = [1; 1; 0; 0; 0; 1; 1; 0]

p = [1 0 0 0 1 1 0 1 1];
q = [1 0 1 1 0 0 0 1];

[quotient remainder] = deconv(p,
q)

z = A(3,:);

for i=1 : 8
    z(i,:) = [A(i,1).*B(1,1),
A(i,2).*B(2, 1), A(i,3).*B(3,
1), A(i,4).*B(4, 1),
A(i,5).*B(5, 1), A(i,6).*B(6,
1), A(i,7).*B(7, 1),
A(i,8).*B(8, 1)];];
end;
z
C
for i=1 : 8
    c(i,:) =
[bitxor(bitxor(bitxor(bitxor(bit
xor(bitxor(bitxor(A(i,1).*B(1,1)
, A(i,2).*B(2, 1)), A(i,3).*B(3,
1)), A(i,4).*B(4, 1)),
A(i,5).*B(5, 1)), A(i,6).*B(6,
1)), A(i,7).*B(7, 1)),
A(i,8).*B(8, 1));];
end;
c

for i=1 : 7
    k(i,:) = [bitxor(S(i,1),
c(i,1));];
end;
k

```

OUTPUT:

433	1	1	1	0	0	0	0	0
410	1	1	1	1	0	0	0	0
Remainder 10111	1	1	1	1	1	0	0	0
Quotient 1	0	1	1	1	1	0	0	0
auxiliary 1	0	0	1	1	1	0	0	0

Remainder 10011	C =
Quotient 10001	
auxiliary 10010	1

Remainder 100	1
Quotient 1	0
auxiliary 10011	0
	0
	0
	1
	1
	0

Remainder 11	
Quotient 100	
auxiliary 1011110	c =

	0
Remainder 1	0
Quotient 1	1
auxiliary 1110001	0
	1
	0
	0
1110001	1
86	0

x = 320	
x = 101000000	k =

-61	
111101	1
94	1
1011110	1
-61	0
	1
	1
	0
	0
z =	0

1	0	0	0	1	0	0	0
1	1	0	0	0	0	0	0

3. Suppose Alice and Bob follow stage 1 and phase 1 of stage 2 of the BB84 protocol, but that they skip phase 2 of stage 2. Also suppose that Eve opaquely eavesdrops on every qubit. Eve also eavesdrops on the stage 2, phase 1 communication between Alice and Bob. Suppose the raw keys consist of k bits each.

- **What is the expected number of bits on which Alice's and Bob's raw keys agree?**

Alice and Bob's raw keys agree on 75% of the bits

- **Explain your answer.**

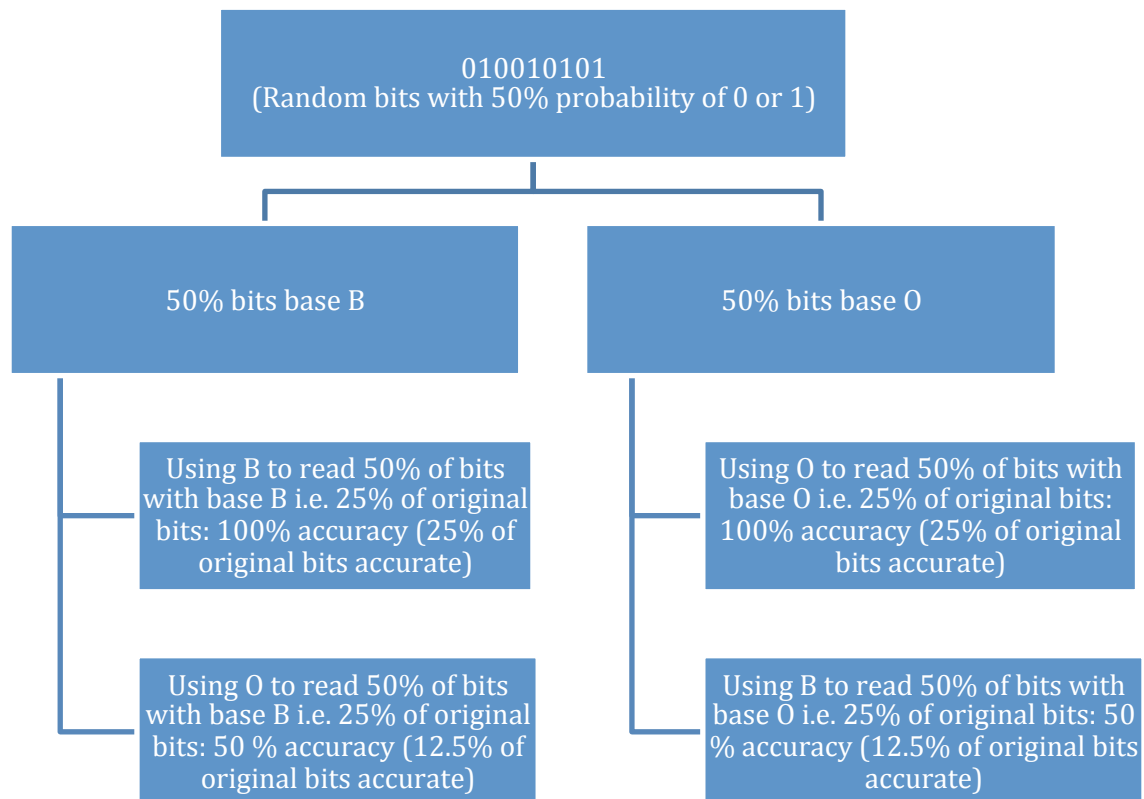
Out of which 50% of the time Alice will use "B" bases and 50% of the time "O" bases. Hence, when Eve is not listening, Bob will choose the right machine 50% of the time. The wrong machine gives the right input 50% of 50% i.e. 25% of the original bits. Hence you get a total accuracy of 75%. So when Bob and Alice decide on the key and they send the qubits using respective bases and Eve eavesdrops she will create an error of 25%. Hence Alice and Bob's raw keys agree on 75% of the bits. Since phase 2 of stage 2 has been skipped they will continue on using the wrong key.

- **What is the expected number of bits of Alice's raw key that Eve will know?**

She(Eve) will know 100% of the key.

- **Explain your answer.**

Since Bob sends the bases over a public channel Eve knows which bases was used for which bits hence when Alice sends the corrected position, which Eve can intercept, she (Eve) will know 100% of the key.



- **Suppose $k = 56$ and Alice uses her raw key to send a sensitive message to Bob using DES. Will Eve be able to decrypt the message in a reasonable amount of time? Eve does not have access to a DES cracker.**

Yes, in this example, Eve will be able to decrypt the message in a reasonable time.

- **Explain your answer.**

Because of the following reasons:

- First, due to the fact that Alice and Bob did not do phase 2 of stage 2, they did not compare results to a point in which they could realized that Eve have intercepted their communication channel. Therefore, they continue on communicating on the same channel without knowing that Eve has been eavesdropping.
- Second, they do not restart the entire quantum operation, because of the previous step, and keep on using a key that Eve has already intercepted which makes it easier for Eve to find out those bits left that did not match to figure out Alice and Bob key.
- Third, Alice and Bob share on the public channel their bits and Alice tells Bob which bits are correct. This allows Eve to figure out the bits left in a very reasonable time.
- Finally, since Bob got about 75% of the correct bits and Eve already got 25% of the correct bits from her eavesdropping. Eve will be able to decrypt the message just as fast as Bob.

- **Suppose $k = 56$ and Alice uses her raw key to send a sensitive message to Bob using DES. Will Bob be able to decrypt the message in a reasonable amount of time? Bob also does not have access to a DES cracker.**

Yes, Bob will be able to decrypt the message in a reasonable time.

- **Explain your answer.**

Eve is communicating with Bob, therefore, in order for they both have a channel using a DES system they both must have the same key since the key used to encrypt the message is the same key used to decrypt the message. Hence, since Bob and Eve have the same key it is possible for them to decrypt the message in reasonable amount of time.

In conclusion, these questions prove that the purpose of quantum cryptography is to detect eavesdropping, as quantum bits are susceptible to change when read.